40 Days of LeetCode Challenge

40 days, 40 problems
DAY :13

LEETCODE PROBLEM NO. 283 : Move Zeros

Problem Link : https://leetcode.com/problems/contains-duplicate/
Arrays · Swapping · Two Pointers · Loops · time-space complexity.

STEP BY STEP SOLUTION

Step 1. Understand the question :-

We are given an integer array nums. We need to move all zeroes to the end while maintaining the relative order of all non-zero elements.

The conditions are:

- Non-zero elements must retain their original order.
- All zeroes must be placed at the end.
- The original array must be modified in-place.
- We should use O(1) auxiliary space.

Step 2. Translating it into simpler language with examples :-

Example 1:
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]

Example 2:
Input: nums = [0]
Output: [0]

Step 3. Identify the pattern

Pattern: Two Pointers — In-place Array Modification

Our solution uses one variable, index, to track the position where the next non-zero element should be placed. We use:

- num: Enhanced for loop variable that accesses each array element.
- index: Tracks the next available position for a non-zero element.
- Unlike the swapping approach, this solution uses direct assignment.

Step 4. Build the logic

Consider the input: nums = [0, 1, 0, 3, 12] Initially: index = 0

First traversal :- Move non-zero elements forward. We traverse the array and check whether each element is non-zero.

- If the element is zero, skip it.
- If it is non-zero, place it at nums[index] and increment index.

After processing all elements: [1, 3, 12, 3, 12]

NOTE:- Why does the array look like this?

Because we have overwritten the first three positions with non-zero values, but the remaining positions still contain their previous values.

Second traversal :- Fill remaining positions with zeroes At this point: index = 3

We fill every position from index 3 until the end of the array with zeroes.

Final array: [1, 3, 12, 0, 0]

Here's the approach we'll follow.

Let's understand the approach before writing Java code.

Consider: nums = [0, 1, 0, 3, 12]

Initially: i = 0 & j = 0

We traverse the array using i.

- If nums[i] is zero, we do nothing.
- If nums[i] is non-zero, we swap it with nums[j] and increment j.

Initial array

0	1	0	3	12
0	1	2	3	4

Encounter 0 — skip it

0	1	0	3	12
0	1	2	3	4

i moves forward; j remains at index 0.
Encounter 1 — swap nums[i] and nums[j]

1	0	0	3	12
0	1	2	3	4

Now j becomes 1.
Encounter 0 — skip it

1	0	0	3	12
0	1	2	3	4

j stays at 1.
Encounter 3 — swap with index 1

1	3	0	0	12
0	1	2	3	4

j becomes 2.
Encounter 12 — swap with index 2

1	3	12	0	0
0	1	2	3	4

Algorithm

1. Initialize index = 0.
2. Traverse the array using an enhanced for loop.
3. For every element:
    - Check whether num != 0.
    - If true, assign num to nums[index].
    - Increment index.
4. After the first loop, all non-zero elements are placed at the beginning.
5. Use a while loop to traverse the remaining positions.
6. Assign 0 to each remaining position.
7. The original array now contains all non-zero elements followed by zeroes.

Pseudocode

```
START

  Initialize index = 0

  FOR each num in nums:

    IF num is not equal to 0:

      nums[index] = num

      index = index + 1

    END IF

  END FOR

  WHILE index is less than nums.length:

    nums[index] = 0

    index = index + 1

  END WHILE

END
```

Step 5. Code in Java

```
class Solution {

    public void moveZeroes(int[] nums) {

        int index = 0;

        for (int num : nums) {

            if (num != 0) {

                nums[index++] = num;  }

        }

        while (index < nums.length) {

            nums[index++] = 0; }

    }

}
```

How to submit

1. Open LeetCode Problem [283 : Move Zeros]
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

A. Class declaration

class Solution {

Defines the class required by LeetCode.

B. Method declaration

public void moveZeroes(int[] nums) {

- public: Makes the method accessible.
- void: The method does not return anything.
- moveZeroes: Name of the method.
- int[] nums: The input integer array.

We use void because the problem requires us to modify the original array directly rather than return a new array.

C. Initialize index

int index = 0;

index represents the position where the next non-zero element should be placed. Initially, it is zero because the first non-zero element should occupy the first position.

D. Enhanced for loop

for (int num : nums) {

This loop traverses every element of the array. Here:

- int num stores the current element.
- nums is the array being traversed.

For example: nums = [0, 1, 0, 3, 12]

num = 0
num = 1
num = 0
num = 3
num = 12

Notice that we don't need an explicit array index to access each element.

E. Check for non-zero elements

if (num != 0) {

Checks whether the current element is non-zero.

- If num == 0, the condition is false, and the element is skipped.
- If num != 0, the condition is true, and we move that element forward.

F. Direct assignment and increment

nums[index++] = num;

- nums[index] = num assigns the current non-zero element to the position indicated by index.
- index++ increments the index by one after using its current value.

The post-increment operator ++ means the current value is used first, and then the variable is increased.For example:

=index = 0;
num = 1;
nums[index++] = num;
This is equivalent to:
nums[index] = num;
index++;
After execution:
nums = [1, 1, 0, 3, 12]
index = 1

The original value at index 0 was zero, and it has now been overwritten with 1.

G. Close the condition and loop

}

The loop continues until every element has been checked. At the end of this loop, all non-zero elements have been moved to the beginning of the array in their original order.

H. Second traversal using while

while (index < nums.length) {

Now we need to fill the remaining positions with zeroes. The condition ensures that the loop runs only while index is within the array boundaries.

For our example:

index = 3
nums.length = 5

Therefore, the loop runs for indices 3 and 4.

I. Assign zero

nums[index++] = 0;

This assigns zero to the current index and then increments index. It works exactly like the previous post-increment operation.

For example:

nums[index] = 0;
index++;

The remaining positions are filled with zeroes.

J. Close the while loop and method

}
}

Once all remaining positions have been filled, the method finishes.

Dry run

Input: nums = [0, 1, 0, 3, 12]

Initial value: index = 0

First loop — Processing non-zero elements

Iteration	num	Condition num != 0	Assignment	index after
1	0	False	Skipped	0
2	1	True	nums[0] = 1	1
3	0	False	Skipped	1
4	3	True	nums[1] = 3	2
5	12	True	nums[2] = 12	3

Array after the first loop: [1, 3, 12, 3, 12]

Second loop — Filling zeroes

Iteration	index before	Assignment	index after
1	3	nums[3] = 0	4
2	4	nums[4] = 0	5

Final array: [1, 3, 12, 0, 0]

Step 7. Time complexity & Auxiliary space complexity

Complexity	Value	Explanation
Time	O(n)	We traverse the array once to move non-zero elements and once to fill zeroes.
Space	O(1)	We use only a constant number of variables and modify the original array.

Note:-

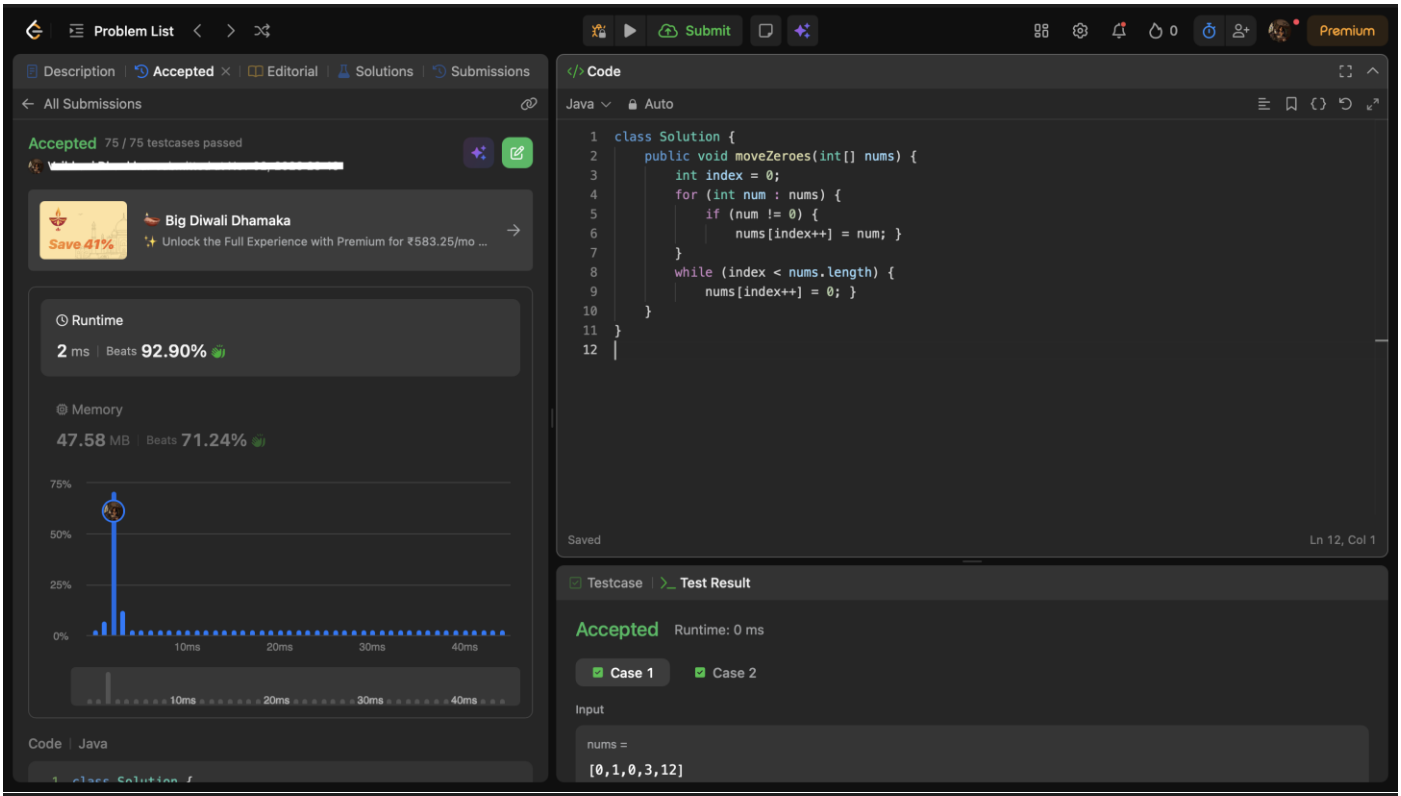
Why is time complexity O(n), not O(2n)?

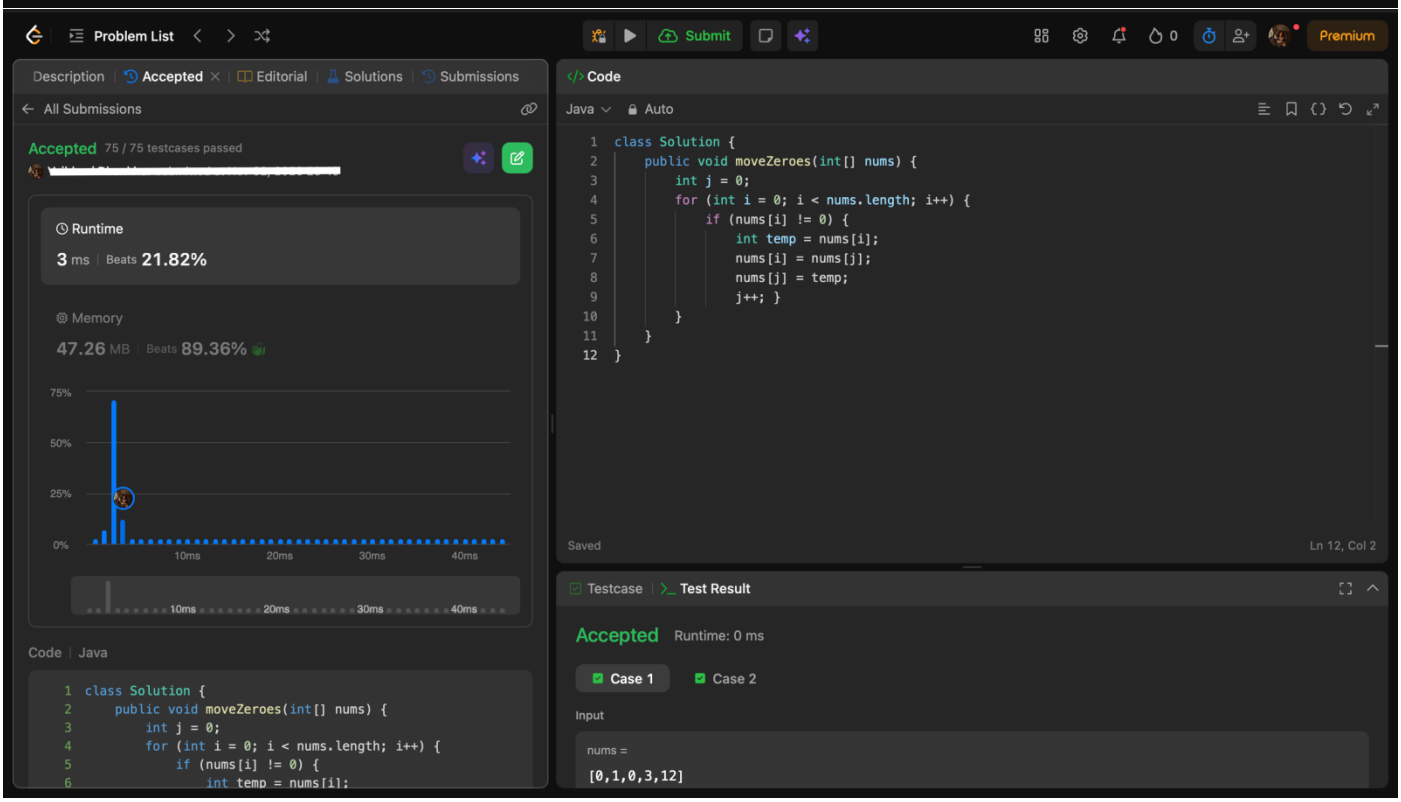
We have two loops, each taking at most O(n) time. [O(n) + O(n) = O(2n) = O(n)] We ignore constant factors when expressing asymptotic complexity.

Why is space complexity O(1)?

We don't create another array, list, or collection. We only use index and num, so the extra space remains constant.

SCREENSHOT:-


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 75 / 75 testcases passed
Big Diwali Dhamaka
Unlock the Full Experience with Premium for ₹583.25/mo ...
Runtime
2 ms | Beats 92.90%
Memory
47.58 MB | Beats 71.24%
Code | Java
Code
Java
Auto
class Solution {
public void moveZeroes(int[] nums) {
int index = 0;
for (int num : nums) {
if (num != 0) {
nums[index++] = num; }
}
while (index < nums.length) {
nums[index++] = 0; }
}
}
Saved
Ln 12, Col 1
Testcase | Test Result
Accepted Runtime: 0 ms
Case 1
Case 2
Input
nums =
[0,1,0,3,12]


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 75 / 75 testcases passed
Runtime
3 ms | Beats 21.82%
Memory
47.26 MB | Beats 89.36%
Code | Java
Code
Java
Auto
class Solution {
public void moveZeroes(int[] nums) {
int j = 0;
for (int i = 0; i < nums.length; i++) {
if (nums[i] != 0) {
int temp = nums[i];
nums[i] = nums[j];
nums[j] = temp;
j++; }
}
}
}
Saved
Ln 12, Col 2
Testcase | Test Result
Accepted Runtime: 0 ms
Case 1
Case 2
Input
nums =
[0,1,0,3,12]